

Hero Skill & Ability System

RFist 기반 스킬 실행 / 쿨타임 / 상태 검증 / 어빌리티 교체 샘플 | 김철규

네트워크 입력 이후 실제 스킬 실행까지 연결되는 시스템을 공개용으로 재구성했습니다. 입력, 상태 조건, 쿨타임, 스킬 실행, 애니메이션/히트박스/VFX 이벤트 연결을 보여주는 것이 목적입니다.

Skill``Ability``Cooldown``Command``State Check

Input

Validate

Cooldown

Execute

Events

샘플 목표

- 스킬 실행 가능 조건 검증
- 쿨타임과 자원 비용 처리
- 영웅 어빌리티별 스킬 세트 교체
- 스킬 실행 이벤트를 애니메이션/히트박스/VFX로 전달

설계 기준

- SkillData: 스킬 정의
- HeroSkill: 실행 단위
- SkillContext: 실행 컨텍스트
- AbilityLoadout: 영웅별 스킬 구성
- SkillRunner: 상태 검증과 실행 흐름

스킬 정의 데이터

```
public enum SkillSlot { Basic, Special, Ultimate }
public enum SkillCostType { None, Stamina, SkillPoint }

public sealed class SkillData
{
    public string Id { get; init; }
    public SkillSlot Slot { get; init; }
    public float Cooldown { get; init; }
    public SkillCostType CostType { get; init; }
    public int CostValue { get; init; }
    public string AnimationTrigger { get; init; }
    public string HitBoxProfileId { get; init; }
    public string EffectId { get; init; }
}
```

정의 데이터

스킬의 쿨타임, 비용, 애니메이션 트리거, 히트박스 프로파일, 이펙트 ID를 데이터로 분리합니다.

장점

스킬 밸런스나 연출 교체가 코드 수정 없이 데이터 변경으로 가능합니다.

스킬 실행 컨텍스트

```
public readonly struct SkillContext
{
    public HeroController Caster { get; init; }
    public IHeroStatus Status { get; init; }
    public Vector3 AimDirection { get; init; }
    public float NetworkLatency { get; init; }
    public bool IsLocalPlayer { get; init; }
}

public readonly struct SkillExecuteResult
{
    public bool Success { get; init; }
    public string Reason { get; init; }

    public static SkillExecuteResult Ok() => new() { Success = true };
    public static SkillExecuteResult Fail(string reason) => new() { Success = false, Reason = reason };
}
```

리뷰 포인트

스킬 실행 시 필요한 입력을 Context 하나로 묶으면 스킬 클래스가 전역 상태나 싱글톤에 의존하지 않습니다.

스킬 실행 단위

```
public abstract class HeroSkill
{
    public SkillData Data { get; }
    private float _nextAvailableTime;

    protected HeroSkill(SkillData data)
    {
        Data = data;
    }

    public SkillExecuteResult TryExecute(SkillContext context)
    {
        if (Time.time < _nextAvailableTime)
            return SkillExecuteResult.Fail("Cooldown");

        if (!CanPayCost(context.Status))
            return SkillExecuteResult.Fail("NotEnoughResource");

        if (!CanExecuteByState(context.Status))
            return SkillExecuteResult.Fail("InvalidState");

        PayCost(context.Status);
        _nextAvailableTime = Time.time + Data.Cooldown;
        ExecuteCore(context);
        return SkillExecuteResult.Ok();
    }

    protected abstract void ExecuteCore(SkillContext context);
}
```

실무 의도

쿨타임, 비용, 상태 검증은 공통 흐름으로 처리하고, 실제 스킬 효과만 하위 클래스에서 구현합니다.

상태 기반 실행 제한

```
private static bool CanExecuteByState(IHeroStatus status)
{
    if (status.IsDead) return false;
    if (status.IsStunned) return false;
    if (status.IsDown) return false;
    if (status.IsCasting) return false;
    return true;
}

private static bool CanPayCost(IHeroStatus status, SkillData data)
{
    return data.CostType switch
    {
        SkillCostType.None => true,
        SkillCostType.Stamina => status.Stamina >= data.CostValue,
        SkillCostType.SkillPoint => status.SkillPoint >= data.CostValue,
        _ => false
    };
}

private static void PayCost(IHeroStatus status, SkillData data)
{
    if (data.CostType == SkillCostType.Stamina)
        status.ConsumeStamina(data.CostValue);
    else if (data.CostType == SkillCostType.SkillPoint)
        status.ConsumeSkillPoint(data.CostValue);
}
```

상태 검증

사망, 스톤, 다운, 캐스팅 중 등 스킬 실행 불가능 상태를 공통 검증합니다.

자원 검증

SP/스태미나 등 비용 정책을 공통화해 스킬마다 중복 구현하지 않습니다.

공격 스킬 구현 예시

```
public sealed class StrikeSkill : HeroSkill
{
    private readonly ISkillEventBus _eventBus;

    public StrikeSkill(SkillData data, ISkillEventBus eventBus) : base(data)
    {
        _eventBus = eventBus;
    }

    protected override void ExecuteCore(SkillContext context)
    {
        _eventBus.Publish(new SkillStartedEvent
        {
            Caster = context.Caster,
            AnimationTrigger = Data.AnimationTrigger,
            EffectId = Data.EffectId
        });

        _eventBus.Publish(new HitBoxRequestedEvent
        {
            Caster = context.Caster,
            ProfileId = Data.HitBoxProfileId,
            AimDirection = context.AimDirection,
            Latency = context.NetworkLatency
        });
    }
}
```

리뷰 포인트

스킬이 애니메이터/히트박스/VFX를 직접 잡지 않고 이벤트를 발행합니다. 연출 시스템과 전투 판정을 느슨하게 연결합니다.

영웅 어빌리티별 스킬 세트

```
public enum HeroAbilityType { Boxer, Ninja, Wizard }

public sealed class AbilityLoadout
{
    private readonly Dictionary<SkillSlot, HeroSkill> _skills = new();

    public HeroAbilityType Type { get; }

    public AbilityLoadout(HeroAbilityType type, IEnumerable<HeroSkill> skills)
    {
        Type = type;
        foreach (var skill in skills)
            _skills[skill.Data.Slot] = skill;
    }

    public bool TryGetSkill(SkillSlot slot, out HeroSkill skill)
        => _skills.TryGetValue(slot, out skill);
}

public sealed class HeroAbilityManager
{
    private AbilityLoadout _current;

    public void ChangeAbility(AbilityLoadout loadout)
    {
        _current = loadout;
        AbilityChanged?.Invoke(loadout.Type);
    }

    public event Action<HeroAbilityType> AbilityChanged;
}
```

실무 의도

캐릭터의 어빌리티가 바뀌면 스킬 구성이 통째로 교체됩니다. 네트워크 동기화 대상은 AbilityType만 두고, 각 클라이언트가 같은 Loadout을 구성할 수 있습니다.

입력에서 스킬 실행까지 연결

```
public sealed class HeroSkillRunner
{
    private readonly HeroAbilityManager _abilityManager;
    private readonly IHeroStatus _status;

    public SkillExecuteResult TryRun(
        SkillSlot slot,
        HeroInputSyncContext syncContext,
        Vector3 aimDirection)
    {
        if (!_abilityManager.Current.TryGetSkill(slot, out var skill))
            return SkillExecuteResult.Fail("SkillNotFound");

        var context = new SkillContext
        {
            Caster = _heroController,
            Status = _status,
            AimDirection = aimDirection,
            NetworkLatency = syncContext.EstimatedLatency,
            IsLocalPlayer = syncContext.IsLocal
        };

        return skill.TryExecute(context);
    }
}
```

NetworkInput과 연결

입력 샘플의 HeroActionRouter가 SkillRunner를 호출하면 입력-스킬 흐름이 완성됩니다.

결과 처리

실패 이유를 반환해 UI 피드백이나 디버깅 로그에 사용할 수 있습니다.

애니메이션/히트박스/VFX 이벤트 연결

```
public interface ISkillEventBus
{
    void Publish<T>(T skillEvent);
    void Subscribe<T>(Action<T> handler);
}

public sealed class SkillAnimationPresenter
{
    public void OnSkillStarted(SkillStartedEvent e)
    {
        e.Caster.Animator.SetTrigger(e.AnimationTrigger);
    }
}

public sealed class SkillHitBoxPresenter
{
    public void OnHitBoxRequested(HitBoxRequestedEvent e)
    {
        var profile = _profileRepository.Find(e.ProfileId);
        _hitBox.Open(profile.CreateHitInfo(e.Caster, e.AimDirection, e.Latency));
    }
}

public sealed class SkillVfxPresenter
{
    public void OnSkillStarted(SkillStartedEvent e)
    {
        _effectPool.Spawn(e.EffectId, e.Caster.transform.position);
    }
}
```

리뷰 포인트

스킬 도메인 로직과 표현 계층을 이벤트로 분리하면 스킬 추가 시 연출 교체가 쉬워집니다.

테스트 가능성과 면접 어필 포인트

테스트 항목

- 쿨타임 중 실행 실패
- 자원 부족 실패
- 스톤/다운 상태 실행 실패
- 어빌리티 교체 시 스킬 세트 변경
- 스킬 시작 이벤트 발생
- 히트박스 요청 이벤트 발생

어필 포인트

- 입력-스킬-히트박스 흐름 설계
- 공통 검증과 개별 스킬 효과 분리
- 어빌리티 기반 로드아웃 교체
- 이벤트 기반 연출 연결
- 네트워크 지연값을 스킬 컨텍스트에 전달

연락처

Portfolio: <https://cosmicgiantkoala.github.io> Email: cosmicgiantkoala@gmail.com